

SOULSIDE AI

TAKE-HOME ASSIGNMENT

Backend Engineer — Event-Driven Meeting Webhook Service

Duration	Tech Stack	Deliverable	Database
5–7 Days	Java / Spring Boot	GitHub Repository	Candidate’s Choice

Overview

You are building an event-driven webhook ingestion service for a video meeting platform using Java Spring Boot. The service will receive webhook events for meeting lifecycle changes and live transcription, process them asynchronously through an event-driven architecture, and persist the results to a database.

This assignment is designed to evaluate how you think about system design, code architecture, and the trade-offs you make when building backend services that need to be reliable, maintainable, and scalable.

Domain Model

The system revolves around three core domain concepts. Understanding the distinction between them is critical to this assignment.

Entity	Description	Key Attributes
Meeting	A recurring or reusable meeting room/entity. A meeting can host many sessions over its lifetime. Think of it as the “container.”	id, title, roomName, organizedBy, createdAt
Session	A specific instance (occurrence) of a meeting. Each time a meeting starts, a new session is created. When the meeting ends, that session is closed.	sessionId, meetingId, status (LIVE / ENDED), startedAt, endedAt
Transcript Segment	An individual chunk of transcribed speech belonging to a session. Multiple segments arrive over the lifetime of a session and must be assembled in order to form the complete transcript.	transcriptId, sessionId, sequenceNumber, speaker, content, startOffset, endOffset

Relationship Summary

Meeting (1) → (*) Session: A single meeting can have many sessions over time.

Session (1) → (*) Transcript Segment: Each session accumulates transcript segments as speech is transcribed in real time.

The `meeting.started` event signals a new session for an existing (or new) meeting. The `meeting.transcript` event streams transcript chunks for the active session. The `meeting.ended` event closes the session.

Scenario

A video meeting platform sends webhook notifications to your service as meetings progress. Your service must handle the following lifecycle:

```
meeting.started → meeting.transcript (1..N chunks) → meeting.ended
```

meeting.started — A new session has begun. Create or update the Meeting entity, and create a new Session with status **LIVE**.

meeting.transcript — A chunk of transcribed speech has arrived. Append it as a Transcript Segment associated with the active session. Segments must be stored in a way that preserves their ordering (via **sequenceNumber**) so the full transcript can be reconstructed after the meeting.

meeting.ended — The session has concluded. Update the Session status to **ENDED** and record the end time.

The final goal: after a meeting ends, a consumer of your API should be able to retrieve the complete, ordered transcript for that session.

Sample Webhook Payloads

meeting.started

```
{
  "event": "meeting.started",
  "meeting": {
    "id": "50c8940e-1b97-402a-97d6-2708b7fec41",
  }
}
```

```
"sessionId": "05e57591-d89e-45c9-ae44-08dc1eaad0e0",
"title": "Meeting title",
"roomName": "lcfvaa-absxch",
"status": "LIVE",
"createdAt": "2022-12-13T06:57:09.736Z",
"startedAt": "2022-12-13T06:57:09.736Z",
"organizedBy": {
  "id": "70c5d391-5bca-4cf3-9907-bec205798adb",
  "name": "Soulside"
}
}
```

meeting.transcript

```
{
  "event": "meeting.transcript",
  "meeting": {
    "id": "50c8940e-1b97-402a-97d6-2708b7feca41",
    "sessionId": "05e57591-d89e-45c9-ae44-08dc1eaad0e0"
  },
  "data": {
    "transcriptId": "alb2c3d4-e5f6-7890-abcd-ef1234567890",
    "sequenceNumber": 42,
    "speaker": {
      "id": "70c5d391-5bca-4cf3-9907-bec205798adb",
      "name": "Soulside"
    },
    "content": "So if we look at the quarterly numbers, the growth rate has been consistent.",
    "startOffset": "34",
    "endOffset": "41",
    "language": "en"
  }
}
```

Transcript Payload Notes

- The meeting object is intentionally minimal — full meeting details were provided in meeting.started.
- sequenceNumber determines ordering. Consider how you would handle duplicates where a transcript chunk can be received twice
- transcriptId uniquely identifies each chunk and can serve as a natural deduplication key.
- startOffset / endOffset are relative to the session start time, not absolute timestamps. Assume the meeting started at 8am. If there is a transcript chunk which got spoken at 5mins into the meeting, startOffset will be 300 seconds (rather than being 8:05 am)

meeting.ended

```
{
```

```
{
  "event": "meeting.ended",
  "meeting": {
    "id": "50c8940e-1b97-402a-97d6-2708b7feca41",
    "sessionId": "05e57591-d89e-45c9-ae44-58dc1eaad0e0",
    "title": "Meeting title",
    "status": "LIVE",
    "createdAt": "2022-12-13T06:57:09.736Z",
    "startedAt": "2022-12-13T06:57:09.736Z",
    "endedAt": "2022-12-13T07:04:37.052Z",
    "organizedBy": {
      "id": "70c5d391-5bca-4cf3-9907-bec205798adb",
      "name": "Soulside"
    }
  },
  "reason": "HOST_ENDED_MEETING"
}
```

Functional Requirements

Webhook Endpoint

- Implement a REST endpoint (e.g., POST /api/webhooks) that accepts the JSON payloads described above.
- Route incoming events based on the event field to the appropriate handler.
- Return an appropriate HTTP response immediately upon receiving the webhook (e.g., 202 Accepted).
- Validate incoming payloads and return meaningful error responses for malformed requests.

Event-Driven Processing

- On receiving a webhook, publish a domain event using any event-driven mechanism of your choice.
- Examples include: Spring Application Events, an in-memory message broker, Kafka, RabbitMQ, or any other approach you find appropriate.
- An event listener/consumer should pick up the event asynchronously and perform the corresponding database operation.

Data Operations

meeting.started: Create the Meeting entity if it does not exist (or update it if it does). Create a new Session entity with status LIVE.

meeting.transcript: Append a new Transcript Segment record to the active session. Preserve sequenceNumber for ordering.

meeting.ended: Update the Session status to ENDED and record the endedAt timestamp.

GET endpoint: Expose at least one read endpoint (e.g., GET /api/meetings/{id}/sessions/{sessionId}/transcript) that returns the complete, ordered transcript for a given session.

Database Layer

- Use Spring Data JPA (or a Spring-compatible alternative) for data access.
- Include entity definitions, repository interfaces, and any necessary migrations or schema setup.
- The application should be runnable with minimal setup (e.g., using an embedded database like H2 for local development is acceptable).

Simulation Script

We have provided a shell script that simulates a basic happy-path meeting lifecycle: a meeting starts, three transcript chunks arrive in order, and the meeting ends. Use this to validate your implementation.

```
#!/bin/bash
# simulate_meeting.sh - Happy-path simulation script
# Usage: ./simulate_meeting.sh [BASE_URL]
#
# This script simulates a basic meeting lifecycle:
#   meeting.started → transcript chunks → meeting.ended

BASE_URL="${1:-http://localhost:8080}"
WEBHOOK_URL="$BASE_URL/api/webhooks"

MEETING_ID="50c8940e-1b97-402a-97d6-2708b7feca41"
SESSION_ID="05e57591-d89e-45c9-ae44-08dc1eaad0e0"
ORGANIZER_ID="70c5d391-5bca-4cf3-9907-bec205798adb"

echo "=== Sending meeting.started ==="
curl -s -X POST "$WEBHOOK_URL" \
  -H "Content-Type: application/json" \
  -d '{
    "event": "meeting.started",
    "meeting": {
      "id": "'$MEETING_ID'",
      "sessionId": "'$SESSION_ID'",
      "title": "Q4 Planning Sync",
      "roomName": "lcfvaa-absxch",
      "status": "LIVE",
      "createdAt": "2024-12-13T06:57:09.736Z",
      "startedAt": "2024-12-13T06:57:09.736Z",
      "organizedBy": {
        "id": "'$ORGANIZER_ID'",
        "name": "Alice Johnson"
      }
    }
  }'
```

```
}
}'
echo ""
sleep 1

# --- Transcript Chunks ---
SPEAKERS=("Alice Johnson" "Bob Smith" "Alice Johnson")
SPEAKER_IDS=(
  "70c5d391-5bca-4cf3-9907-bec205798adb"
  "82d6e402-6cdb-5df4-a018-19ed2fbce1bc"
  "70c5d391-5bca-4cf3-9907-bec205798adb"
)
CONTENTS=(
  "Alright, let us get started with the Q4 planning."
  "Sure. I have the revenue projections ready to share."
  "Great, go ahead and walk us through the numbers."
)
START_OFFSETS=("00:00:02.100" "00:00:05.800" "00:00:09.400")
END_OFFSETS=("00:00:05.200" "00:00:08.900" "00:00:12.600")

for i in 0 1 2; do
  SEQ=$((i + 1))
  TRANSCRIPT_ID=$(uuidgen 2>/dev/null || cat /proc/sys/kernel/random/uuid)
  echo "=== Sending transcript chunk #${SEQ} ==="
  curl -s -X POST "$WEBHOOK_URL" \
    -H "Content-Type: application/json" \
    -d "{
    \"event\": \"meeting.transcript\",
    \"meeting\": {
      \"id\": \"${MEETING_ID}\",
      \"sessionId\": \"${SESSION_ID}\"
    },
    \"data\": {
      \"transcriptId\": \"${TRANSCRIPT_ID}\",
      \"sequenceNumber\": ${SEQ},
      \"speaker\": {
        \"id\": \"${SPEAKER_IDS[$i]}\",
        \"name\": \"${SPEAKERS[$i]}\"
      },
      \"content\": \"${CONTENTS[$i]}\",
      \"startOffset\": \"${START_OFFSETS[$i]}\",
      \"endOffset\": \"${END_OFFSETS[$i]}\",
      \"language\": \"en\"
    }
  }"
  echo ""
  sleep 1
done

echo "=== Sending meeting.ended ==="
curl -s -X POST "$WEBHOOK_URL" \
  -H "Content-Type: application/json" \
  -d '{
  "event": "meeting.ended",
  "meeting": {
    "id": "${MEETING_ID}",
    "sessionId": "${SESSION_ID}",
```

```
"title": "Q4 Planning Sync",
"status": "LIVE",
"createdAt": "2024-12-13T06:57:09.736Z",
"startedAt": "2024-12-13T06:57:09.736Z",
"endedAt": "2024-12-13T07:04:37.052Z",
"organizedBy": {
  "id": "'$ORGANIZER_ID'",
  "name": "Alice Johnson"
}
},
"reason": "HOST_ENDED_MEETING"
}'
echo ""

echo "=== Simulation complete ==="
echo "Verify: GET $BASE_URL/api/meetings/$MEETING_ID"
```

 Build Your Own Test Scenarios

The script above covers only the happy path. We expect you to create additional test scenarios to exercise edge cases. Think about situations like: duplicate transcript chunks, out-of-order delivery, transcript arriving after meeting.ended, a meeting.ended event for a session that was never started, and concurrent sessions for the same meeting. How your application handles these cases (and how you document your decisions) is part of the evaluation.

Evaluation Criteria

We will evaluate your submission across the following dimensions. There are no trick questions — we want to understand how you think, not just what you code.

Criterion	What We Look For
Application Code Design	Clean, idiomatic Java/Spring Boot code. Meaningful naming, consistent structure, and appropriate use of design patterns. Code should be easy to read and reason about.
Separation of Concerns	Clear boundaries between layers (controller, service, event, repository). Business logic should not leak into controllers, and infrastructure concerns should be isolated from domain logic.
Scalability & Performance	Demonstrate awareness of how the application would behave under increased traffic. Document or implement considerations such as async processing, connection pooling, back-pressure handling, or how you would swap in an external message broker.
Event-Driven Architecture	Thoughtful application of EDA principles. We want to see that you understand why event-driven patterns are used and where they add value — not just that you can wire up an event publisher.

Design Considerations

Any additional decisions you make around error handling, idempotency, retry logic, logging, configuration management, or testing strategy. Bonus points for a brief design document or README section explaining your trade-offs.

Deliverables

1. A GitHub repository (public or private, with access shared) containing the complete source code.
2. A README.md that includes: instructions to build and run the project, a brief architecture overview, the design decisions and trade-offs you made, and any assumptions.
3. Your own test scenarios (scripts, test classes, or Postman collection) beyond the provided simulation script.
4. (Optional) A short design document or architecture diagram showing how the system components interact.

**Important Notes**

- You do NOT need to deploy the application. A locally runnable project is sufficient.
- You do NOT need to integrate a production message broker (Kafka, RabbitMQ, etc.), though you may discuss how you would if scaling beyond in-process events.
- Focus on quality over quantity. A clean, well-structured solution with thoughtful documentation is preferred over a feature-rich but messy codebase.

Bonus (Not Required)

The following are strictly optional, but will be positively noted if present:

- Unit and/or integration tests demonstrating key behaviors.
- Docker Compose setup for easy local execution.
- Use of an external message broker (Kafka, RabbitMQ) with documented rationale.
- Idempotency handling for duplicate webhook deliveries (e.g., using transcriptId for deduplication).
- Structured logging and observability considerations.
- API versioning or webhook signature verification.
- A GET endpoint that reconstructs and returns the full transcript in reading order.

Timeline & Submission

Please complete and submit your solution within 5–7 days of receiving this assignment. If you need additional time, please let us know — we are happy to accommodate reasonable requests.

When you are ready, share the repository link (or invite us as collaborators if the repository is private) along with any additional notes or context you would like us to consider.



Questions?

If anything is unclear or you have questions about the scope, requirements, or expectations, please don't hesitate to reach out. We'd rather you ask than make assumptions that take you in the wrong direction.

Good luck! The Soulside AI team looks forward to reviewing your work.